

Desafio de performance: Custo de ordenação

Assignment: Trabalho Prático

Prof. Fabrício Valadares

Faculdade da Saúde e Ecologia Humana (FASEH)
Rua São Paulo, 958, Parque Jardim Alterosa – CEP 33200-000 – Vespasiano – MG – Brasil

Ana Clara Fagundes do Carmo
Emily Carolina Leocadio Peres da Silva

Resumo

Este trabalho apresenta a implementação e análise comparativa dos algoritmos Selection Sort e Heap Sort, pertencentes às classes de complexidade $O(n^2)$ e $O(n \log n)$, respectivamente. Os algoritmos foram implementados manualmente utilizando arrays e testados com arquivos contendo 1.000 e 1.000.000 de elementos nos cenários de melhor caso, caso médio e pior caso. Os resultados demonstraram uma diferença significativa de desempenho entre os algoritmos, evidenciando a superioridade do Heap Sort para grandes volumes de dados.

Palavras-chave: ordenação, Selection Sort, Heap Sort, análise de algoritmos.

Abstract

This paper presents the implementation and comparative analysis of the Selection Sort and Heap Sort algorithms, which belong to the $O(n^2)$ and $O(n \log n)$ complexity classes, respectively. The algorithms were manually implemented using arrays and tested with datasets containing 1,000 and 1,000,000 elements under best-case, average-case, and worst-case scenarios. The results demonstrated a significant performance difference between the algorithms, highlighting the superiority of Heap Sort for large datasets.

Keywords: sorting, Selection Sort, Heap Sort, algorithm analysis.

1. Introdução

No trabalho implementamos dois algoritmos de ordenação utilizando arrays: o Selection Sort, pertencente ao grupo dos algoritmos simples de complexidade $O(n^2)$, e o Heap Sort,

pertencente ao grupo dos algoritmos eficientes de complexidade $O(n \log n)$. O objetivo foi comparar o desempenho desses algoritmos em diferentes cenários de entrada, analisando seus tempos de execução para arquivos contendo 1.000 e 1.000.000 de elementos nos casos de melhor, médio e pior situação.

2. Funcionamento dos Algoritmos

2.1 Selection Sort

O Selection Sort é um algoritmo de ordenação simples baseado na seleção do menor valor do vetor para primeira posição, depois o segundo menor valor para a segunda posição. Em cada iteração, o algoritmo percorre os elementos restantes procurando o menor valor e realiza uma troca com a posição atual.

Passos do algoritmo:

1. Considera-se a primeira posição do vetor.
2. Procura-se o menor elemento entre os elementos ainda não ordenados.
3. O menor elemento encontrado é trocado com o elemento da posição atual.
4. O processo é repetido até que todos os elementos estejam ordenados.

Apesar de sua simplicidade, o algoritmo apresenta complexidade $O(n^2)$.

2.2 Heap Sort

O Heap Sort utiliza uma estrutura denominada Heap Máxima (Max Heap), na qual o maior elemento permanece na raiz da árvore. Cada elemento da estrutura adaptada deve ficar no formato de um par.

Devido à utilização da estrutura heap, o algoritmo possui complexidade $O(n \log n)$, sendo melhor para grandes volumes de dados.

3. Resultados

Tempos de execução dos algoritmos Selection Sort e Heap Sort.

Arquivo	Selection Sort (ms)	Heap Sort (ms)
dados_melhor_1k.csv	2	2
dados_medio_1k.csv	7	0
dados_pior_1k.csv	1	0
dados_melhor_1M.csv	259.474	92
dados_medio_1M.csv	296.681	157
dados_pior_1M.csv	1.128.699	132

Olhando a tabela, conseguimos analisar que os tempos de execução dos dois tipos de algoritmos utilizados são similares para os arquivos pequenos, porém são significativamente diferentes quando o volume de dados aumenta.

4. Análise Crítica

O Selection Sort possui complexidade $O(n^2)$, fazendo com que o número de operações cresça proporcionalmente ao quadrado da quantidade de elementos. Como consequência, o tempo de execução aumenta em arquivos maiores.

Quando executamos o arquivo, o Selection Sort atingiu mais de 1.128.699 ms no pior caso para 1.000.000 de elementos. Já o Heap Sort concluiu a mesma tarefa em apenas 132 ms.

Esses valores apresentados demonstram que algoritmos $O(n \log n)$ são muito mais eficientes para aplicações que processam grandes volumes de dados.

5. Conclusão

Neste trabalho utilizamos os algoritmos Selection Sort e Heap Sort. Os experimentos demonstraram que a complexidade computacional influencia diretamente o desempenho dos algoritmos de ordenação.

Embora os resultados de tempo para conjuntos pequenos, o Heap Sort mostrou desempenho melhor para grandes volumes de dados. Dessa forma, conseguimos concluir que algoritmos com complexidade $O(n \log n)$ são mais eficientes para aplicações reais que exigem rapidez e escalabilidade.

Referências

- [Overleaf – SBC Template for Papers](#)
- Sociedade Brasileira de Computação – Templates para artigos e capítulos de livros
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Introduction to Algorithms*. MIT Press.
- SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. Addison-Wesley.
- https://www.kelvinsantiago.com.br/ordenacao-com-selection-sort-insertion-sort-bubble-sort-merge-sort-quick-sort-heap-sort-em-java/#google_vignette